

700105 Simulation & Concurrency



UNIVERSITY
of **HULL**

The story so far

The story so far

- Updating State
 - Calculating linear forces and torques
 - Gravity
 - Calculating new velocities and positions (particle/projectile motion and rigid bodies)
 - Integration methods (Euler, Semi-Implicit Euler, Verlet, Runge-Kutta)
- Collisions - Geometric Objects (Shapes)
 - Primitives (point, line, line segment, plane, sphere, cylinder, capsule, cuboid, triangle, convex hull)
 - Methods (Inside-Outside, Intersects, Contains, SAT, GJK)
- Collision Response
 - Calculating new velocities or forces after collision using impulses or forces
 - Conservation of linear and angular momentum (including inertia)

Convex Shapes

What is a convex shape?

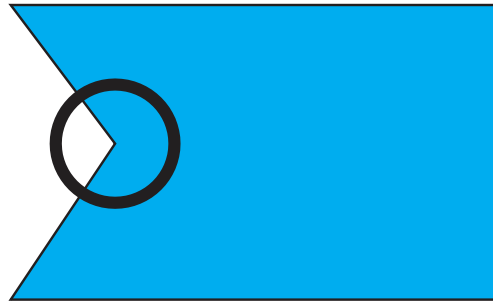
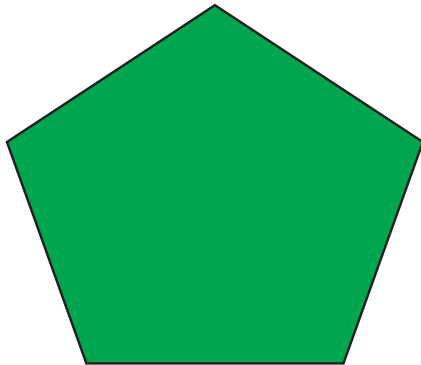
- A **convex shape** is a shape where, for any two points inside the shape, the line segment connecting those points lies entirely within the shape.
- **No Indentations:** The shape does not curve inward or have "dents."
- **Line Segment Rule:** If you pick any two points within the shape, the straight line between them stays inside the shape.
 - Useful property for things like navigation meshes
- **Internal Angles:** In a convex polygon, all interior angles are less than or equal to 180°
 - This is linked to cross sign consistency (coming later)

What is a convex shape?

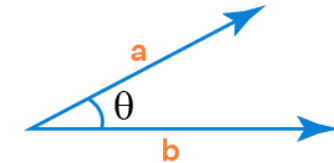
- Convex shapes make collisions calculations less complicated
- Only need to check if points or edges overlap
- No need to worry about things like
 - Holes
 - Indentations
 - one object wrapping around another

Checking for Convexity - 2D Angle Method

- Check to see if all the internal angles are less than 180 degrees



Angle Between Two Vectors
Formulas



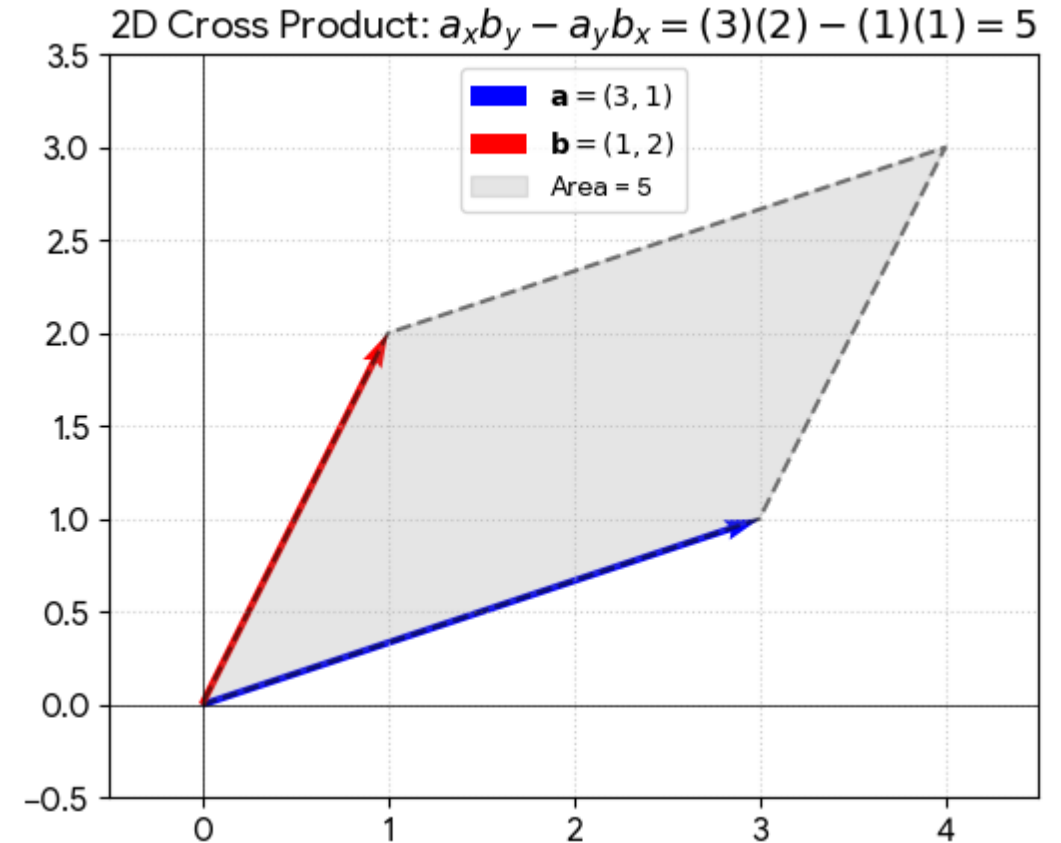
$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|}$$

(OR)

$$\sin \theta = \frac{|\mathbf{a} \times \mathbf{b}|}{|\mathbf{a}| |\mathbf{b}|}$$

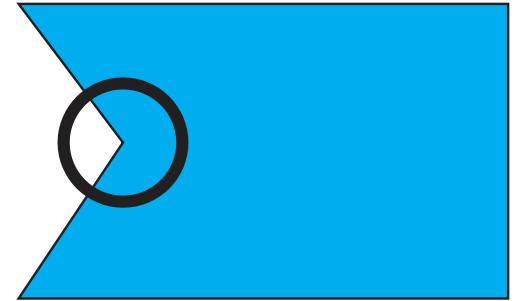
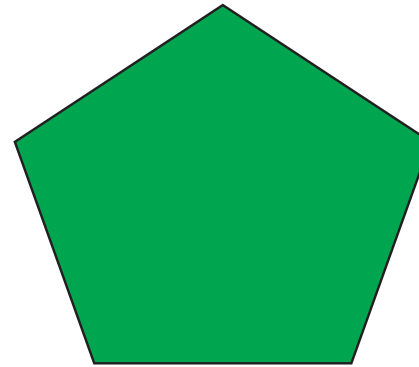
Checking for Convexity - 2D Cross Product Method/Plane method

- Make sure every vertex is the same side of each line
- 2D cross product $a \times b = a_x b_y - a_y b_x$
- Gives the **signed** area of the parallelogram formed by vectors a and b
- The sign represents the direction of rotation from a to b
 - +ve – b is anticlockwise rotation from a
 - -ve – b is clockwise rotation from a
 - 0 – a and b are collinear (they are parallel or anti-parallel)



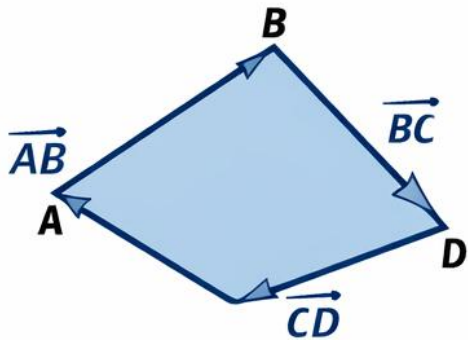
Checking for Convexity - 2D Cross Product Method/Plane method

- In a convex shape all



Checking Convexity Using the 2D Cross Product (Plane Method)

Calculate Cross Products:

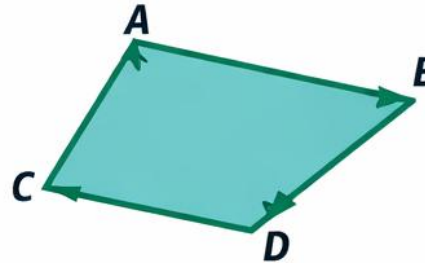


$$\vec{AB} \times \vec{BC} = Z_1$$

$$\vec{BC} \times \vec{CD} = Z_2$$

$$\vec{CD} \times \vec{DA} = Z_3$$

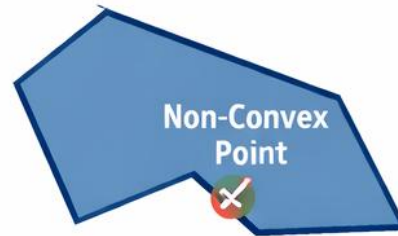
Convex Shape:



✓ All $Z_1, Z_2, Z_3 > 0$ or < 0 ✓

✓ Same Sign for All Cross Products ✓

Concave Shape:



✗ Mixed Signs in Cross Products

✗ Z_1, Z_2, Z_3 not consistent

All Same Sign

Shape is Convex ✓

Mixed Signs

Shape is Concave ✗

Extend to 3D

- In 3D compare each face (typically a triangle)
- For each face
 - Calculate the outward facing normal vector $n = (B - A) \times (C - A)$
 - Project every other point P onto that face using the dot product $n \cdot (P - A)$
 - Ensure all values are positive or all values are negative

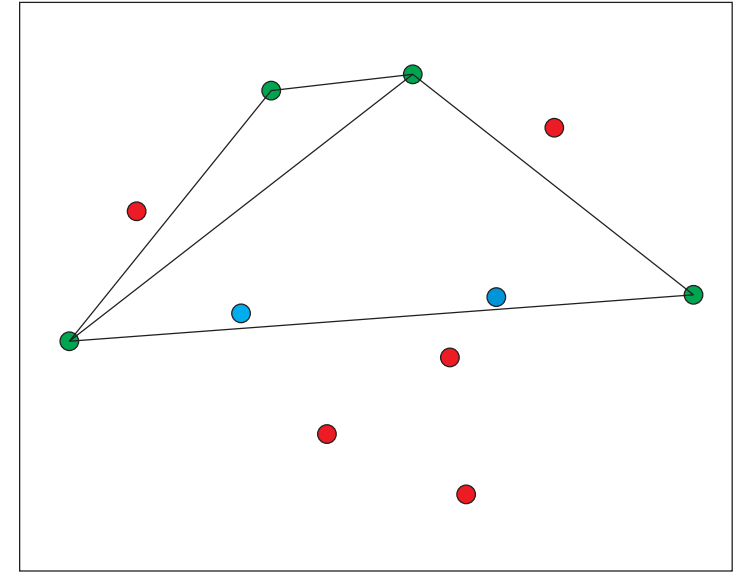
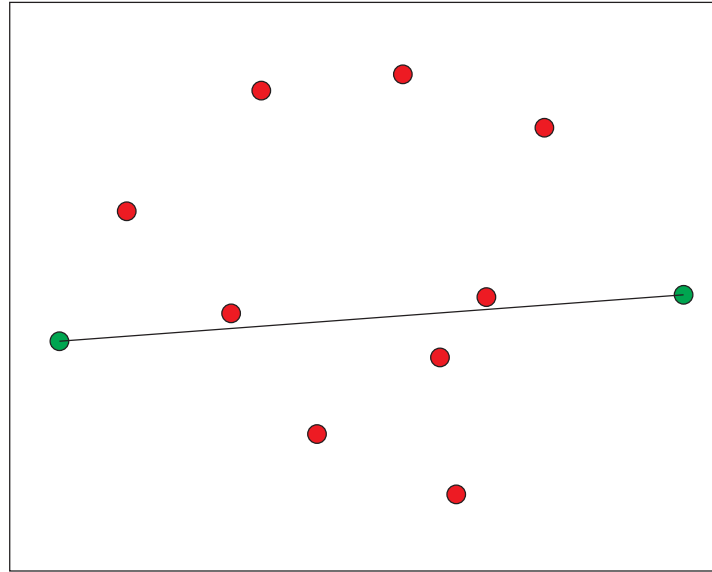
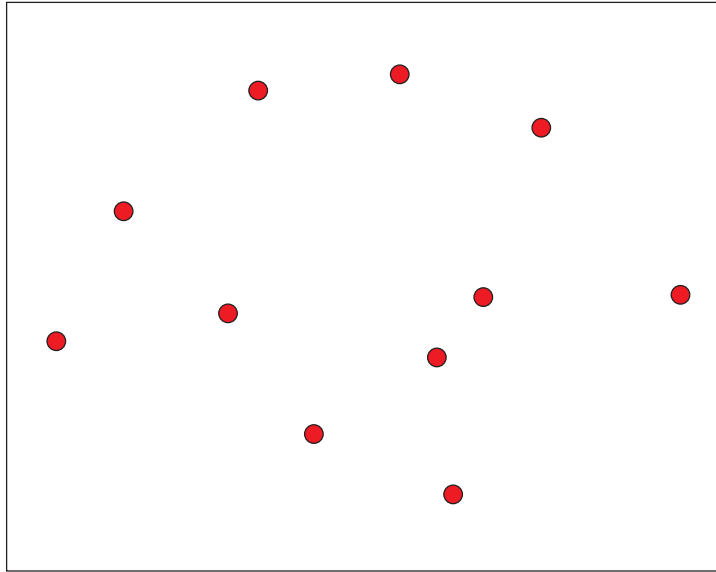
Finding a Convex Hull

- Sometimes you might have a concave shape and you want to calculate a convex hull for that object.
 - Note, if you use this for collision detection you will end up with false positives where the original shape is concave.

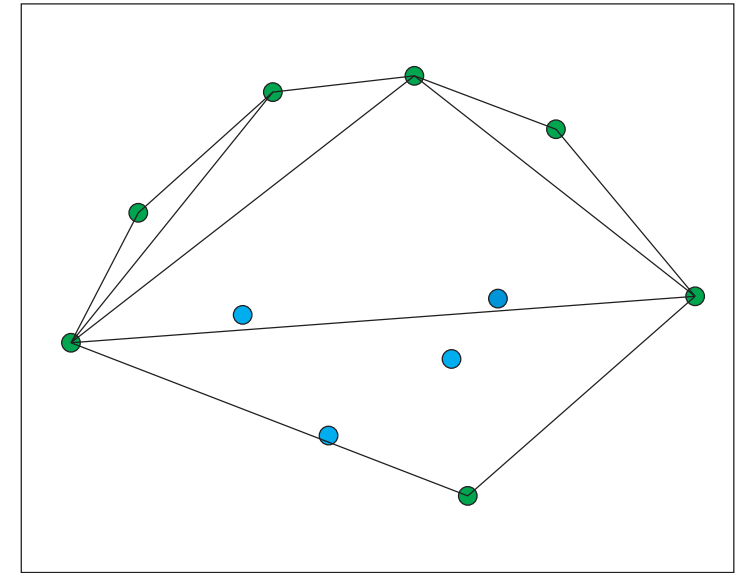
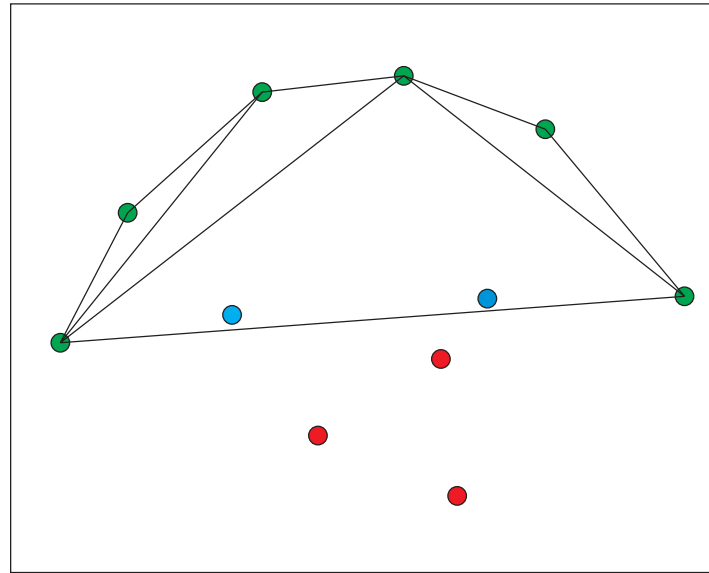
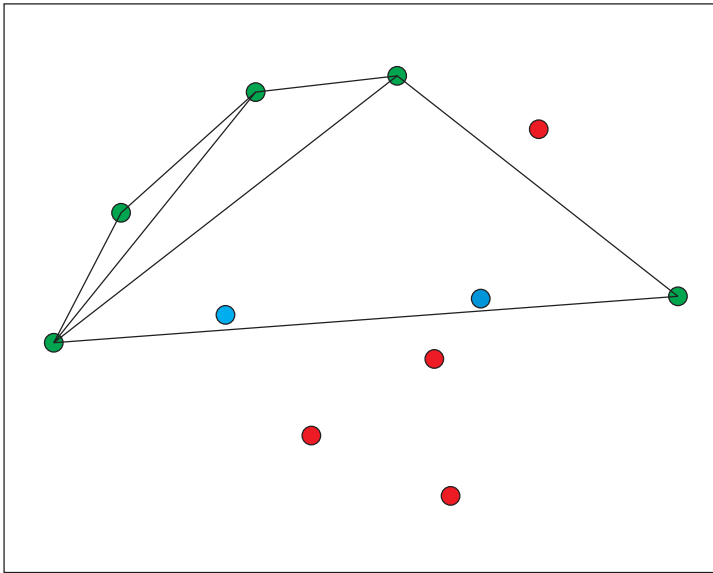
QuickHull Algorithm

- Goal from a set of points determine the points that form the outside edge
- Find the two points furthest from each other in a direction (usually x)
 - These are definitely part of the convex hull
- Find the point furthest from those two points
 - This is part of the convex hull, and forms a triangle with the other two points
 - None of the points inside that triangle are part of the convex hull
- Continue with each new edges until all points are either part of the hull or not
- **Note this will not result in points in the correct order**

QuickHull Algorithm

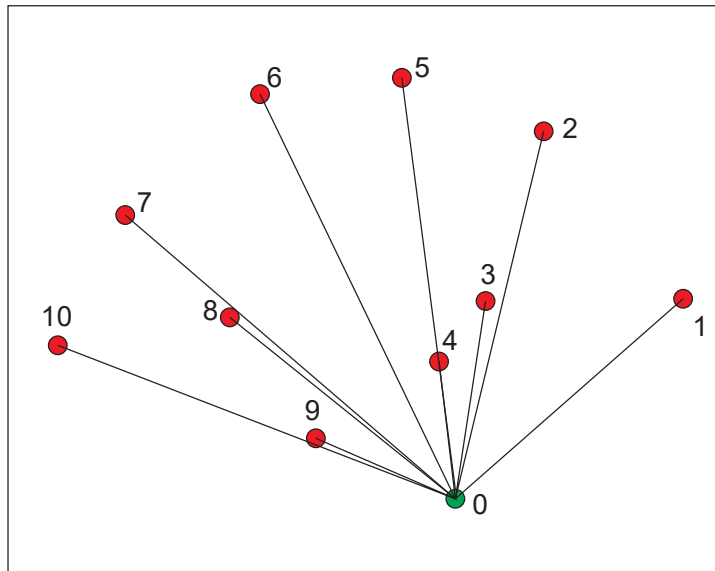


QuickHull Algorithm



Graham Scan Convex Hull Algorithm

- Start with an extreme point that is definitely part of the convex hull
- Calculate the angle to each other point and sort the result
- Add candidates in checking the angle at each point with the last point on the stack
- If the internal angle is < 180 add it to the stack
- If not pop the last point of the stack and compare to the previous point



0, 1
0, 1, 2
0, 1, 2, 3
0, 1, 2, 4
0, 1, 2, 5
0, 1, 2, 5, 6
0, 1, 2, 5, 6, 7
0, 1, 2, 5, 6, 7, 8
0, 1, 2, 5, 6, 7, 8, 9
0, 1, 2, 5, 6, 7, 10

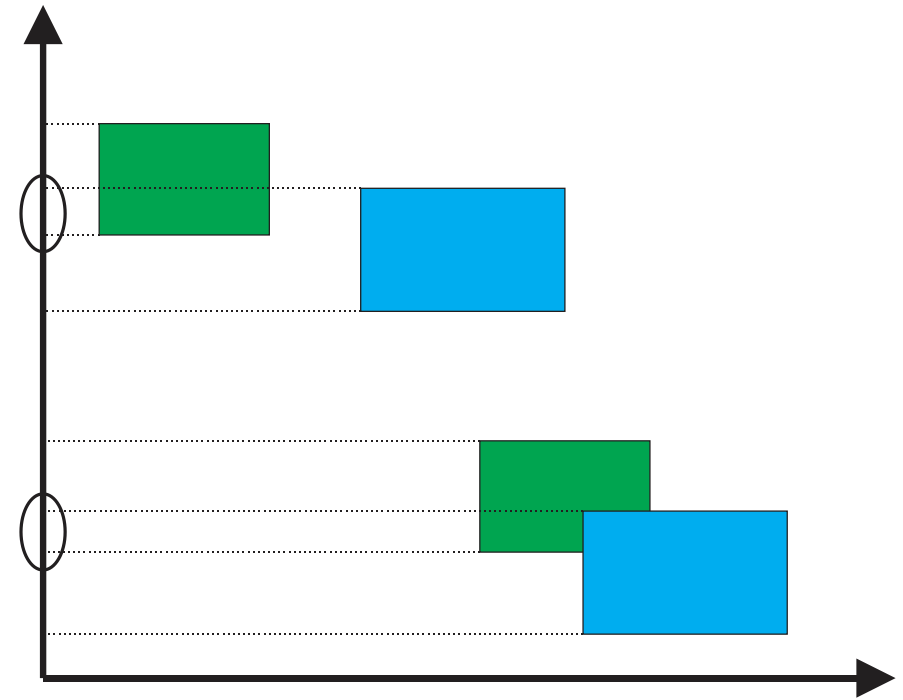
Separated Axis Theorem

Separated Axis Theorem

- Two convex shapes are ***not*** colliding there will be at least one axis where their projections do not overlap
- If such an axis exists the shapes are separated – hence “Separated Axis Theorem”

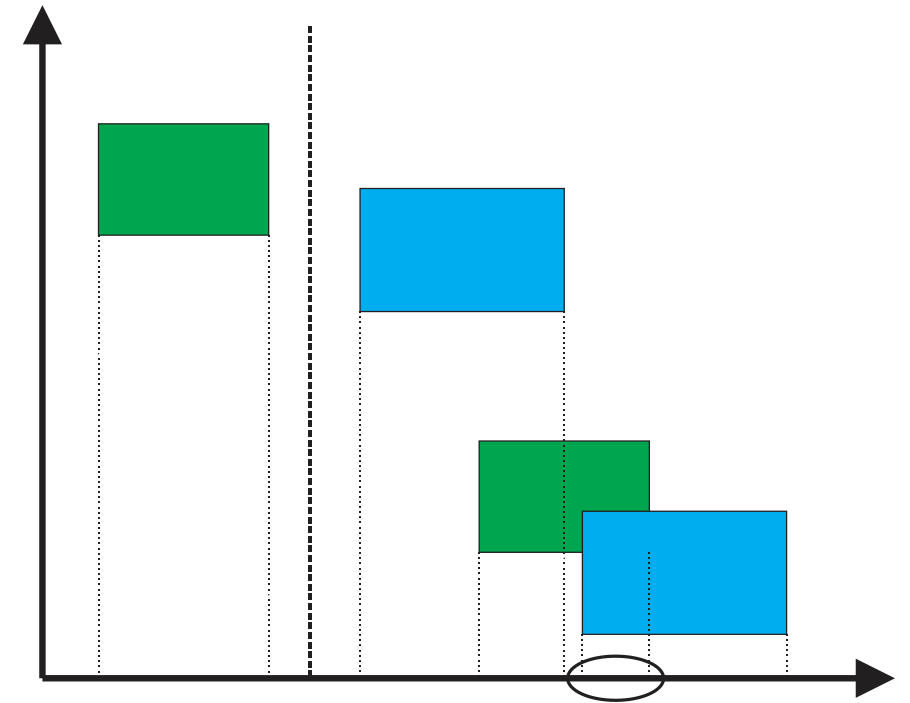
Start with AABB

- Project the points from each box onto each axis and see if they overlap
- Here the projected points on the y axis overlap so the x axis is ***not*** a separating axis



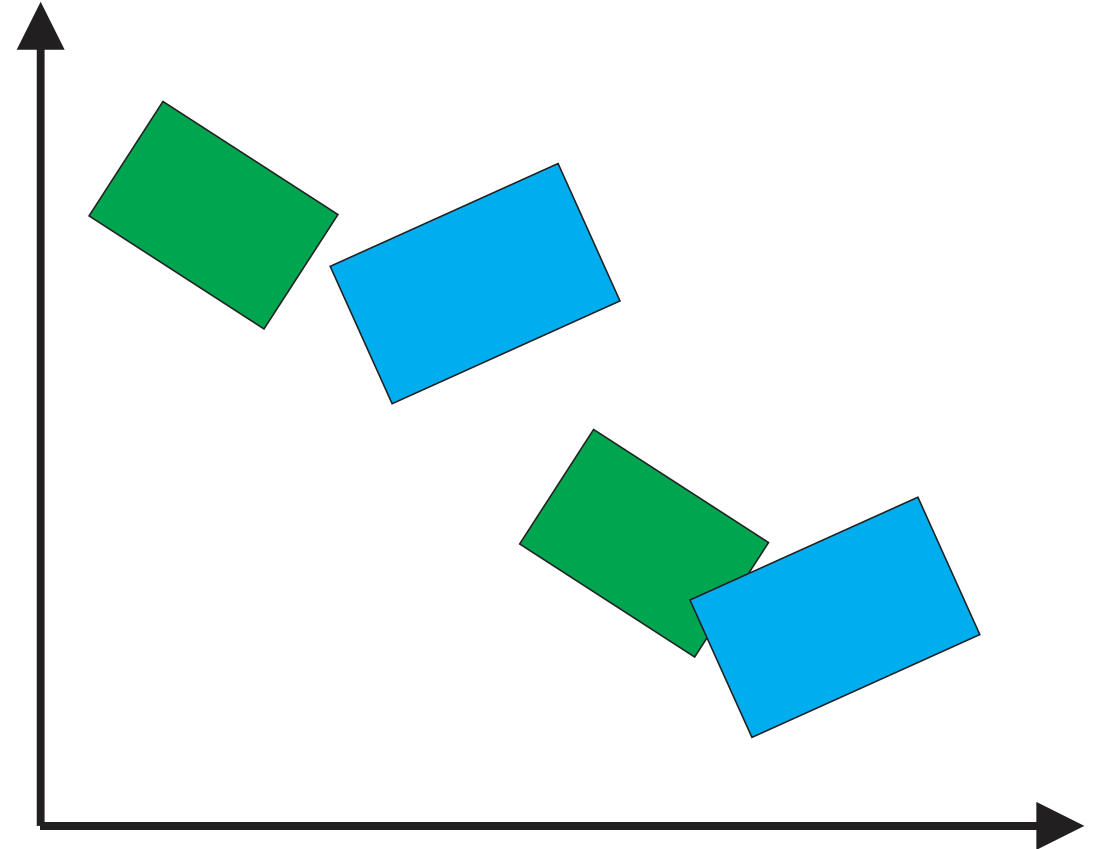
Start with AABB

- Here the projected points on the x axis overlap do **not** overlap for the first pair of AABBs so the y axis **is** a separating axis
- For the second pair of objects there is an overlap so there is no separating axis, and



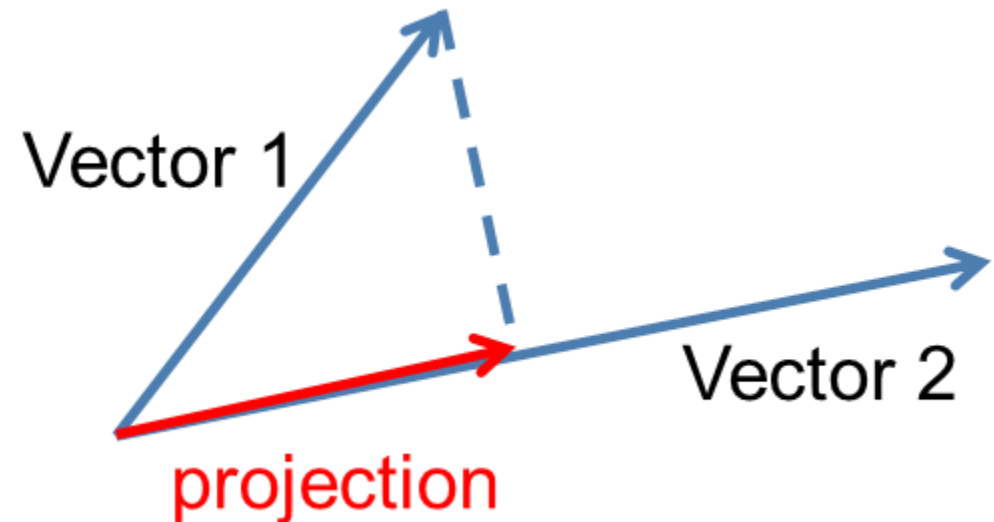
Now Consider OOBB

- The AABB example works because we know the boxes are axis aligned, so we know which axis to pick
- What if we have a rotated box instead?



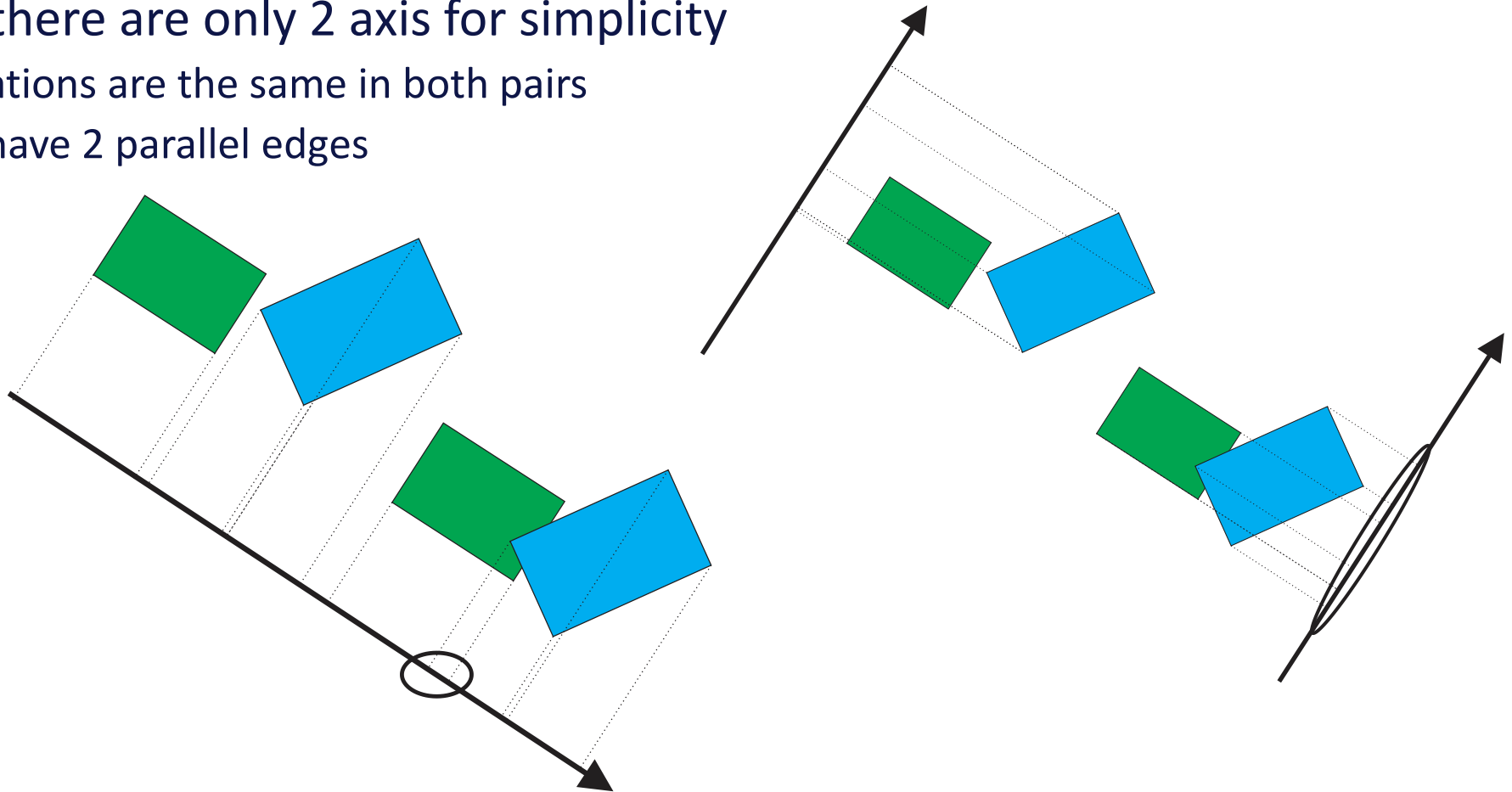
Projecting Points

- You can find the projection of a point on a general axis using the dot product
- V_1 projected on to V_2 is $\frac{v_1 \cdot v_2}{|v_2|^2} v_2$



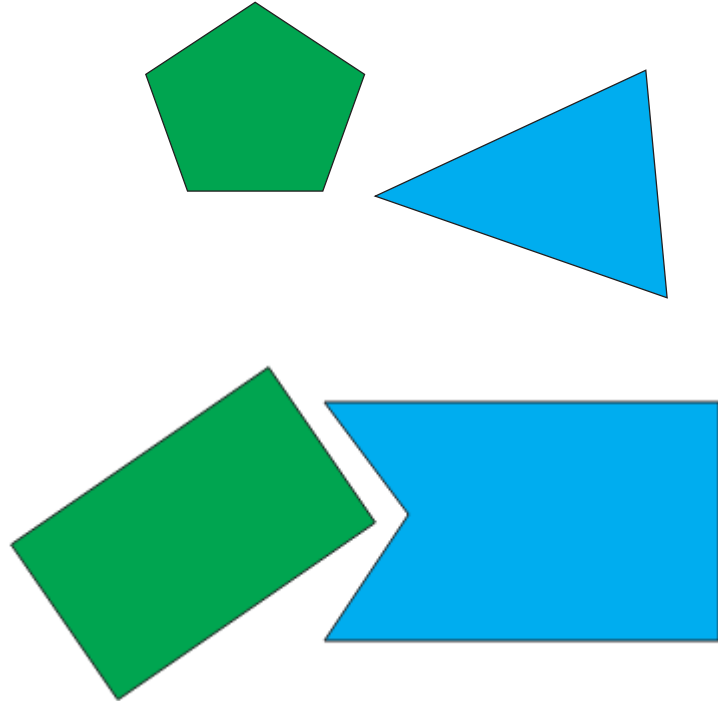
Choosing axis to test

- To pick axis to test just use axis in the direction of the normal vectors of both shapes
- In the diagram there are only 2 axis for simplicity
 - The boxes rotations are the same in both pairs
 - The boxes all have 2 parallel edges

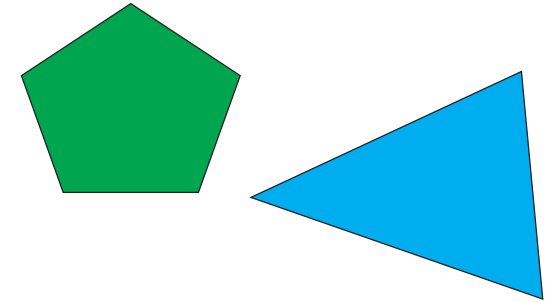


Now Consider a General Polygon

- This works for any pair of **convex** polygons
- This won't always work if either shape are concave



SAT Summary



- For each edge of shape A
 - Project all vertices from shape A onto the normal of that edge and find the min and max
 - Project all vertices from shape B onto the normal of the edge and find the min and max
 - If there is no overlap then we have found separating axis – return false
- For each edge of shape B
 - Project all vertices from shape A onto the normal and find the min and max
 - Project all vertices from shape B onto the normal and find the min and max
 - If there is no overlap then we have found separating axis – return false
- There is no separating axis – return true

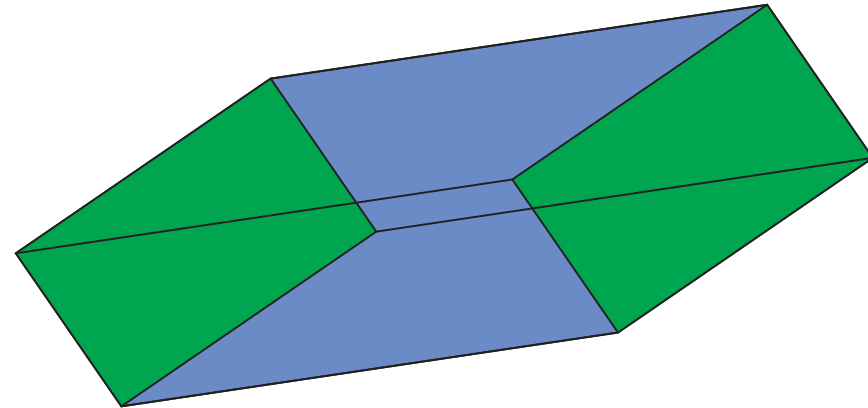
Now extend to 3D

- To extend separated axis theorem to 3D
- Look for separating plane instead of separating axis
- Project vertices onto each plane face and check to see if projections overlap

Gilbert-Johnson-Keerthi Algorithm

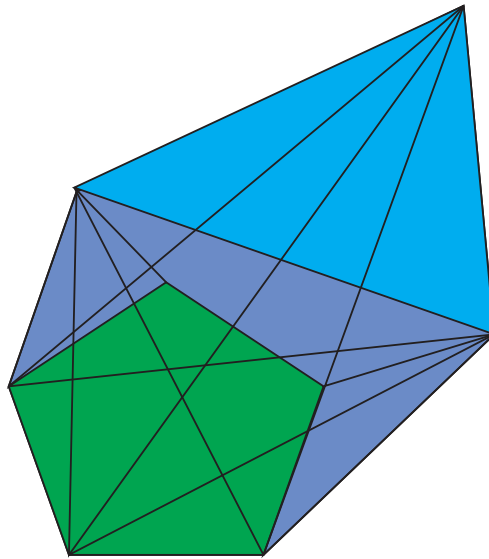
Minkowski Sum

- Named after Hermann Minkowski
- Minkowski Sum is the shape defined by the addition of each point of one shape to each point on another shape
- This idea can be useful for continuous collision detection – adding the shape of an object in one frame to the shape of the object in the next frame
 - Sometimes known as a “swept shape”



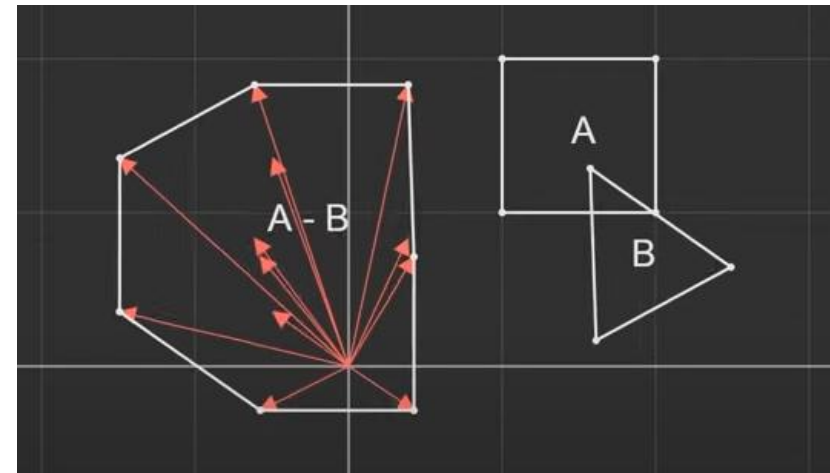
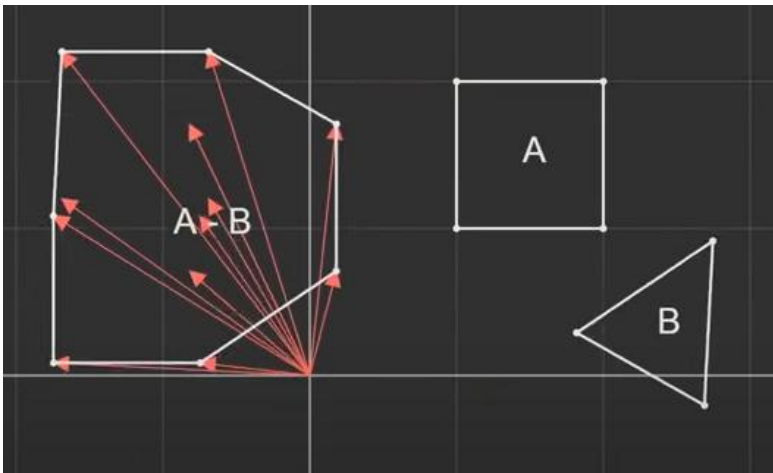
Minkowski Sum with different objects

- For different objects there may be different vertices, so the Minkowski sum is obtained by adding every vertex in one object to every vertex in the other object!



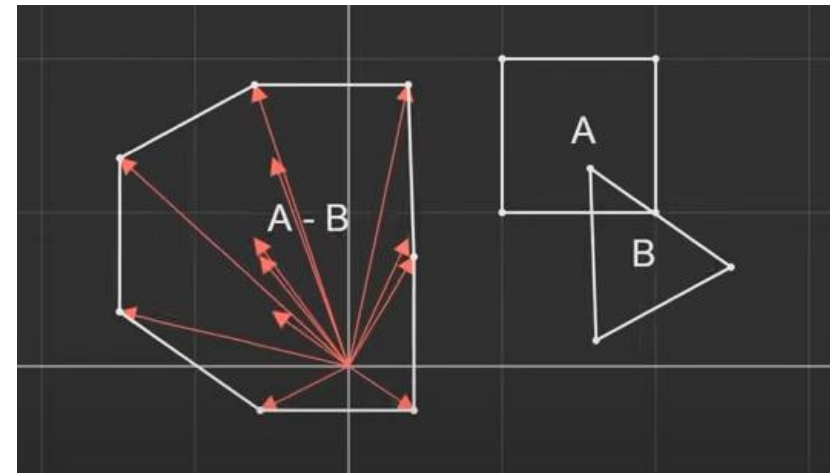
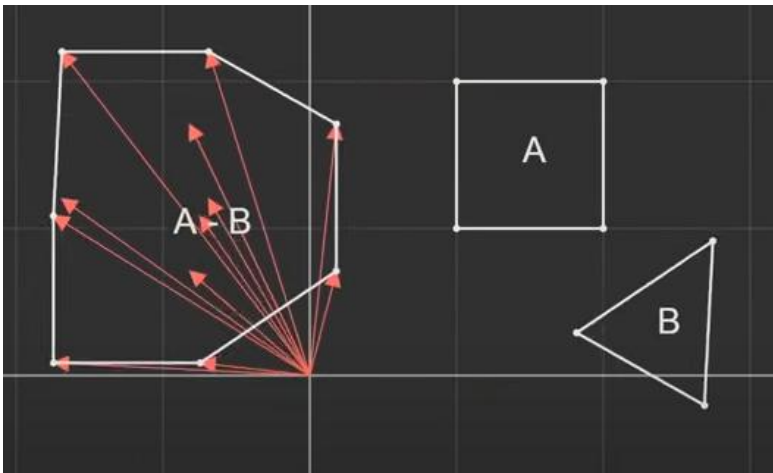
Minkowski Difference

- The Minkowski Difference is obtained by subtracting every point in one shape from every point in the other
- The Minkowski Difference has the property that if the origin is inside that shape the two shapes were intersecting



Minkowski Difference

- The Minkowski Difference is obtained by subtracting every point in one shape from every point in the other
- The Minkowski Difference has the property that if the origin is inside that shape the two shapes were intersecting

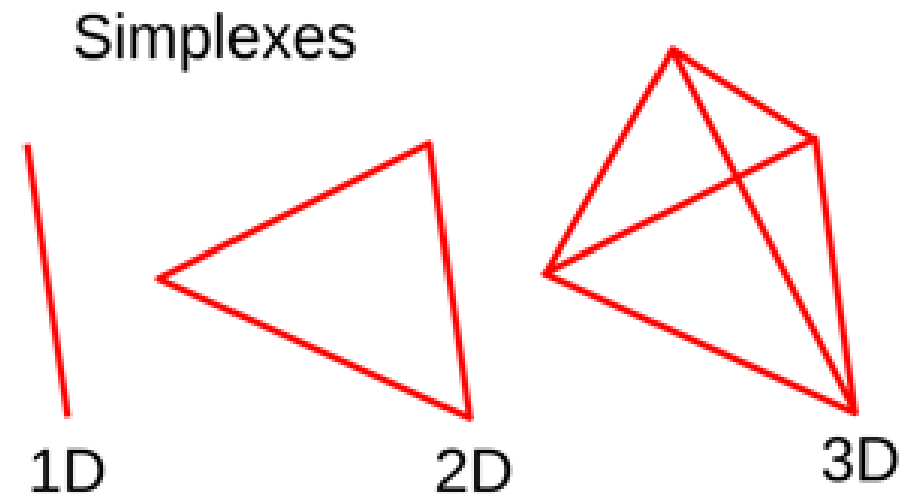


Gaining efficiency

- Many of the algorithms we have talked about so far involve calculations for every face against every vertex.
- This is far too inefficient.
- Two approaches
 - Broad phase collision detection
 - GJK algorithm avoids computing the entire Minkowski difference

Gilbert-Johnson-Keerthi Algorithm

- The Minkowski Difference is very expensive to calculate
- The GJK algorithm is a sequence of steps to reduce the computation
- The goal is to find the ***simplex*** that contains the origin as fast as possible
 - A simplex is the simplest polytype for a given number of dimensions – so a triangle in 2d space and a tetrahedron in 3d space



Support Points

- The furthest point in any direction is called the “support point”
- This has the advantage that we can collide any shape against any other shape
 - Triangle, circle, rectangle, polygon, whatever
- As long as we can find a support point this works for a 3d mesh and ***any other shape (e.g. sphere)*** – not just polygon polygon intersections

GJK Algorithm

- Find support points on two shapes in a direction and subtract to get the corresponding point in the Minkowski difference (A)
- Pick the direction from that point towards the origin and get the support points from each shape – subtract to get the point on the Minkowski difference (B)
- Check to see if that point is past the origin – if not then shapes are not colliding – return false
- Next pick the point perpendicular to the line from A to B facing the origin to get point C on the Minkowski difference and check to see if origin is inside triangle ABC
- If so return true
- If not choose edge of ABC that is closest to the origin
- Get support points in normal direction towards the origin
- If that point is already on the triangle then we ran out of appropriate triangles, so no collision

GJK Independent Learning Resources

- <https://www.youtube.com/watch?v=MDusDn8oTSE>
- <https://www.youtube.com/watch?v=ajv46BSqcK4>

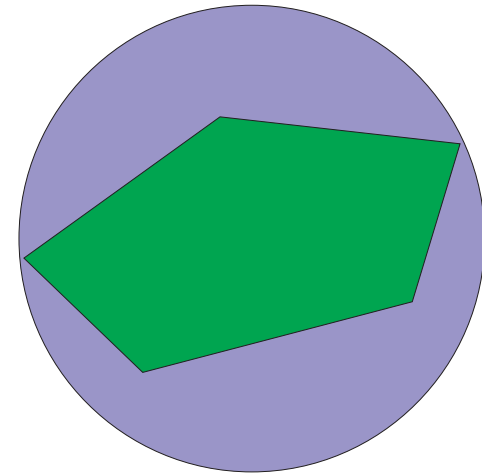
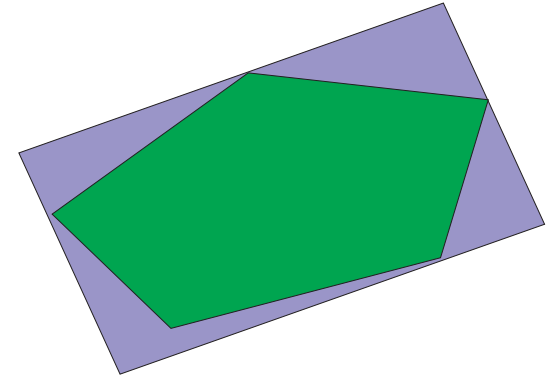
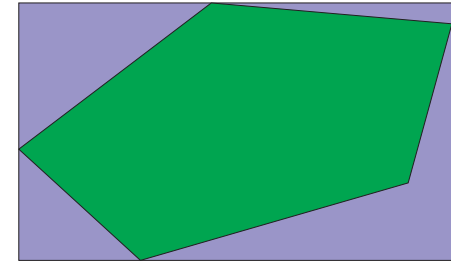
Broad Phase and Narrow Phase

Broad Phase and Narrow Phase Collision Detection

- Narrow phase methods are accurate but expensive
 - Triangle and Polygon Intersection
 - Separating Axis Theorem
 - GJK Algorithm
- Broad phase methods are inaccurate but cheap
 - Typically bounding shape tests
 - Bounding Spheres
 - Axis Aligned Bounding Boxes are good

Broad Phase Trade Off

- Axis Aligned Bounding Box
- Object Oriented Bounding Box
 - What if object's orientation changes?
- Bounding Sphere
- Not always about minimising space
- Trade off between computation time and number of hits



Saptial Segmentation

Spatial Segmentation Techniques

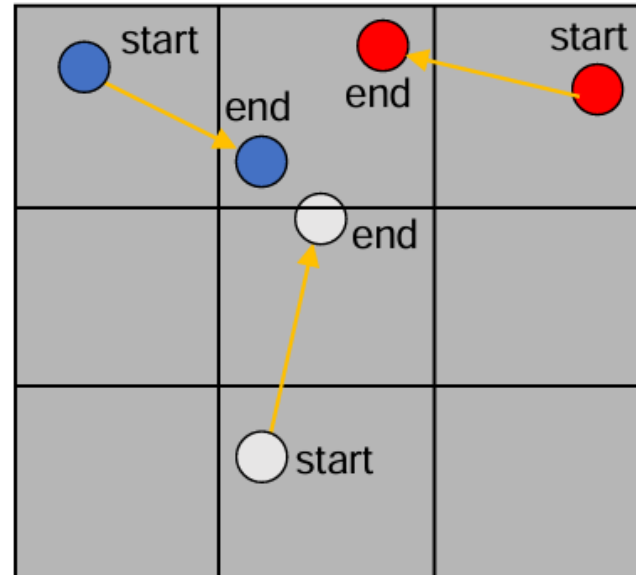
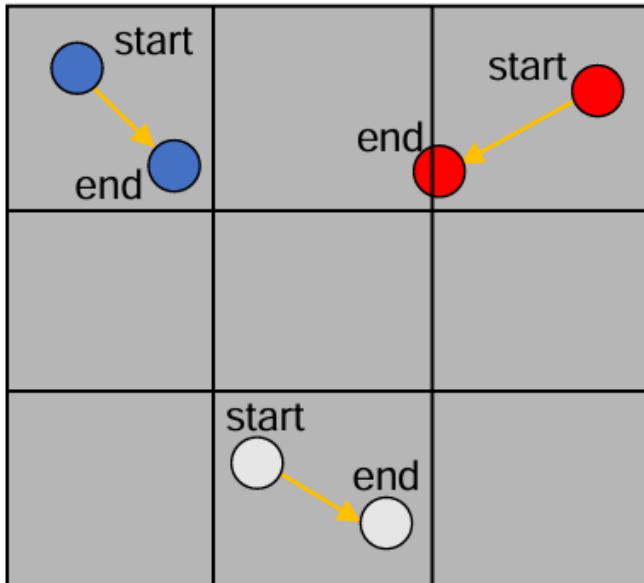
- If we consider an object against all other objects, then we will be calculating $O(n^2)$ possible collisions
- We can use spatial sorting so that only nearby objects are tested for possible collisions

Spatial Segmentation Techniques

- Simple Grid
- Quadtree/Octree (Hierarchical “Grids”)
- BSP Tree
- K-D Tree

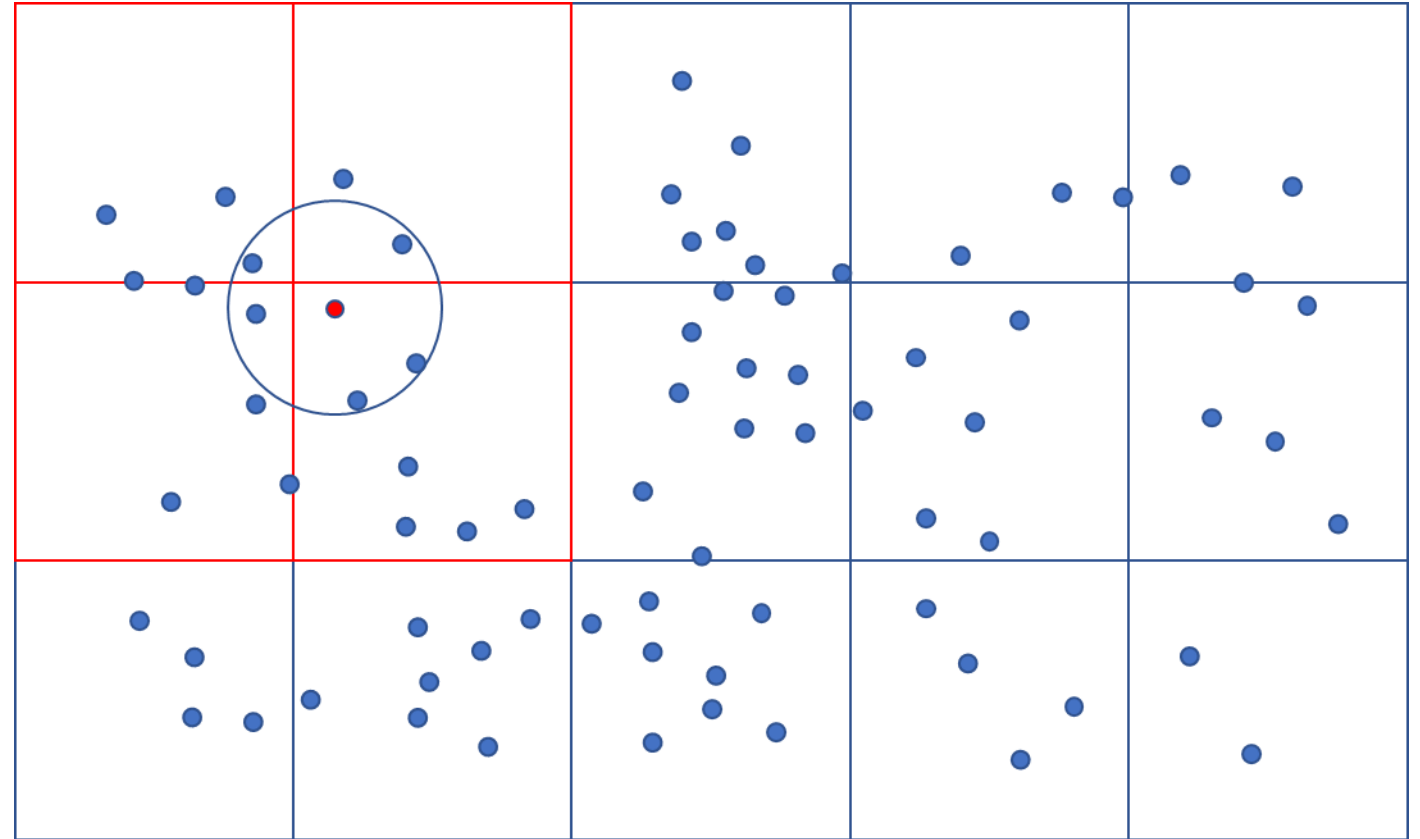
Regular Grid

- Split up world into a regular grid
- Each cell maintains a set of objects that are in that cell
- Choice of maximum object size and maximum object speed is important



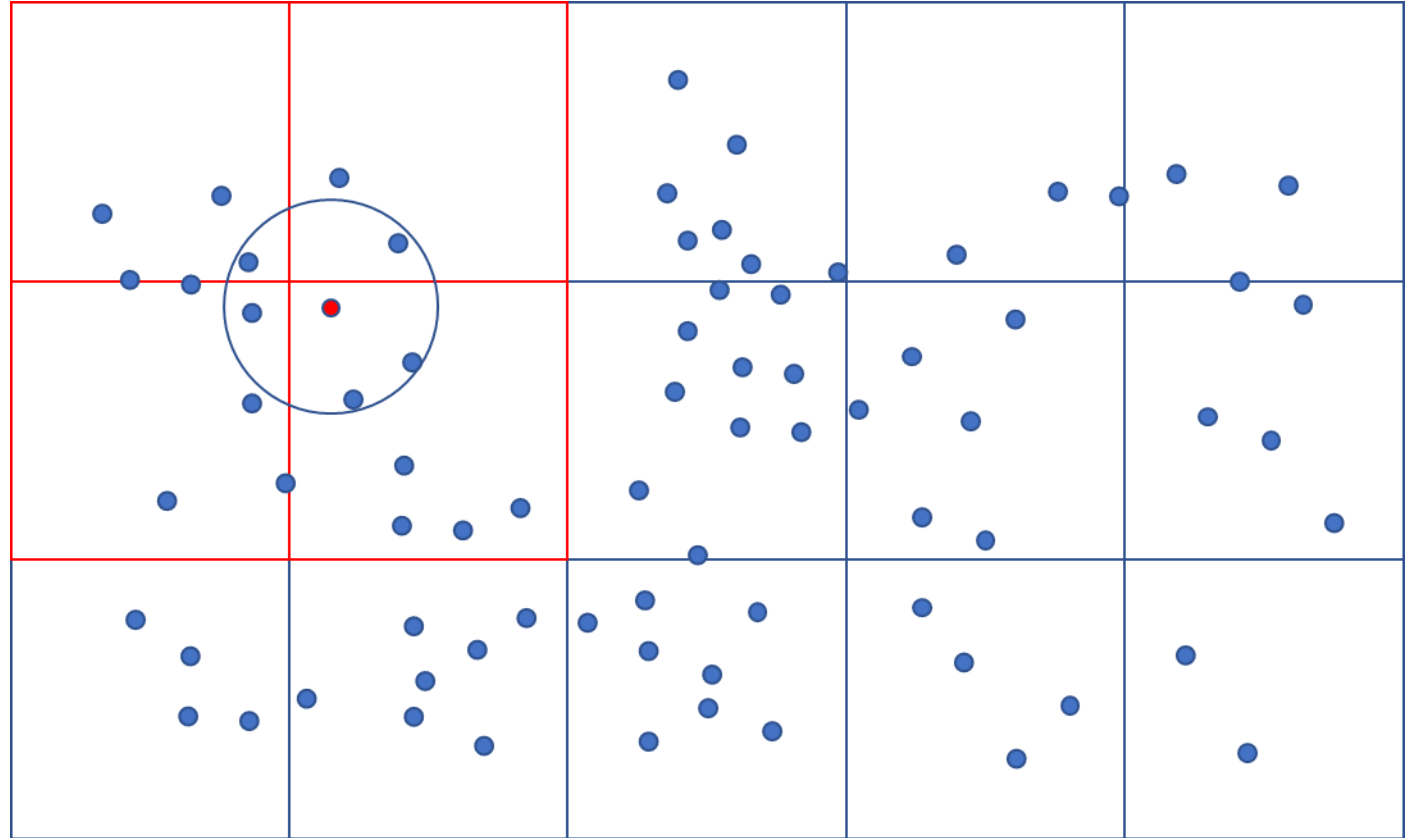
Regular Grid

- Can limit the search for neighbours significantly
- Need neighbourhood radius to be smaller than grid size
- Calculate which corner the agent is in
- Test against agents in the same cell and three adjacent cells



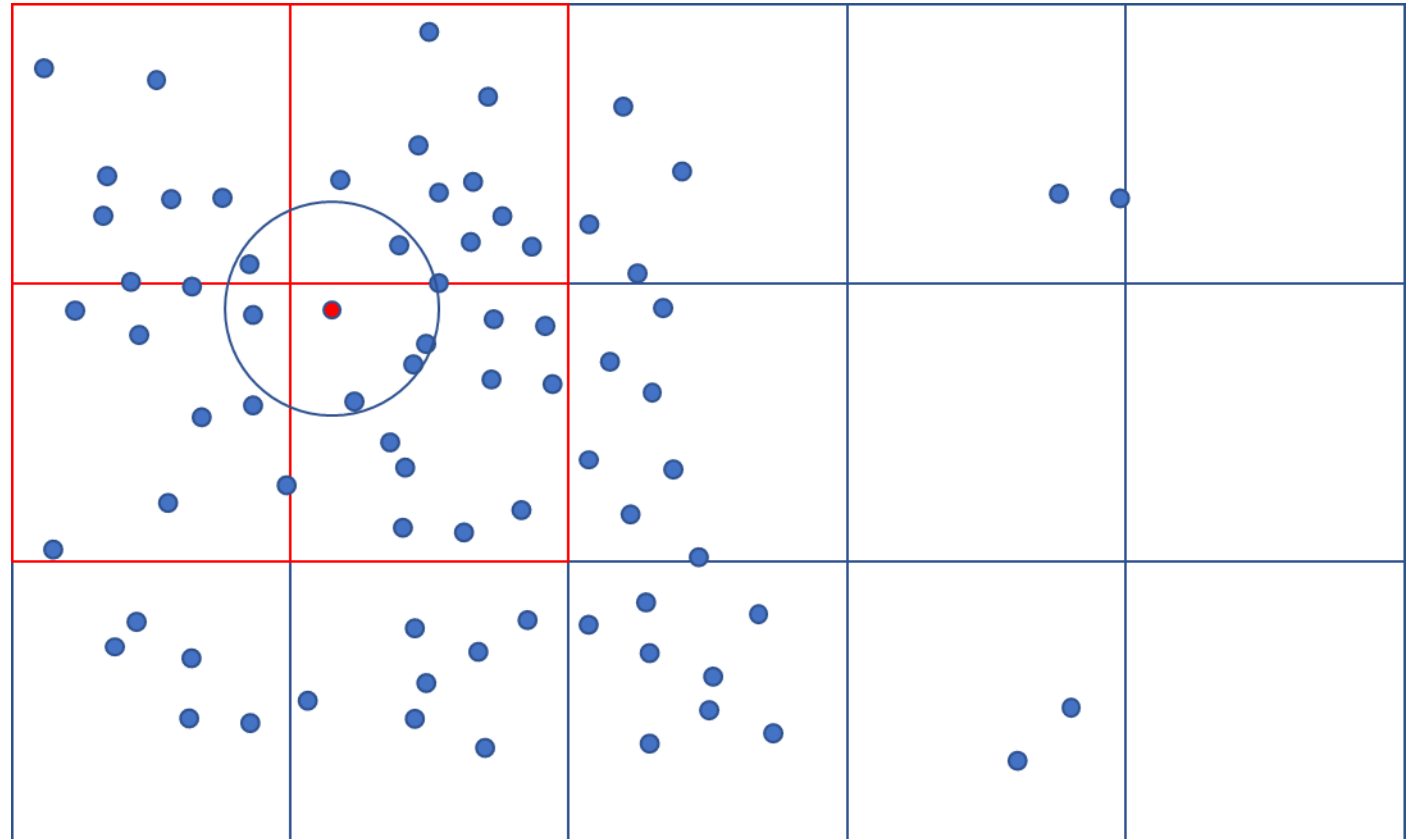
Regular Grid

- In this (flocking) example we test against just 17 other agents (instead of ~75) to find the neighbourhood of just 5
- Again - Choice of maximum object size and maximum object speed is important



Problem – non uniform distribution of stuff

- We can use a hierarchy structure, like a quadtree (in 2d) or an octree (in 3d)
 - You can still use a quadtree in a 3d environment. Imagine is most of the things are on the ground!



BSP Tree

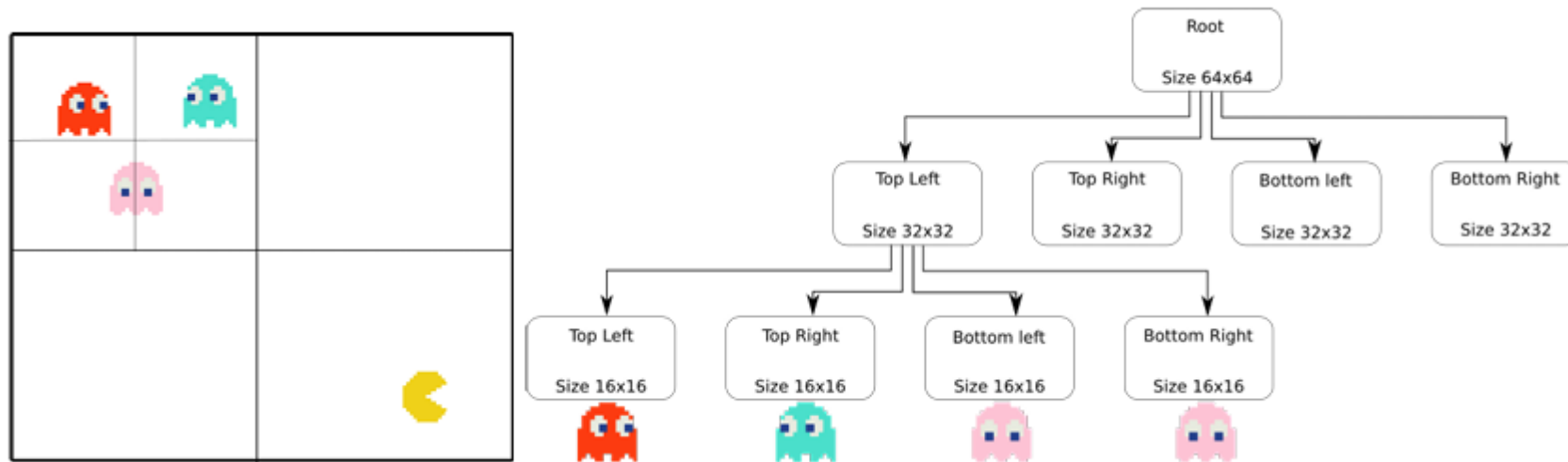
Binary Space Partition Tree

- Made famous by doom
- A tree structure that splits larger space in two (hence binary)
- Each nodes has two parents, and are split by a plane
- A nice bi-product of this method is that because every region is split by a plane every region must be convex
- Unlike grids the split is driven by level geometry

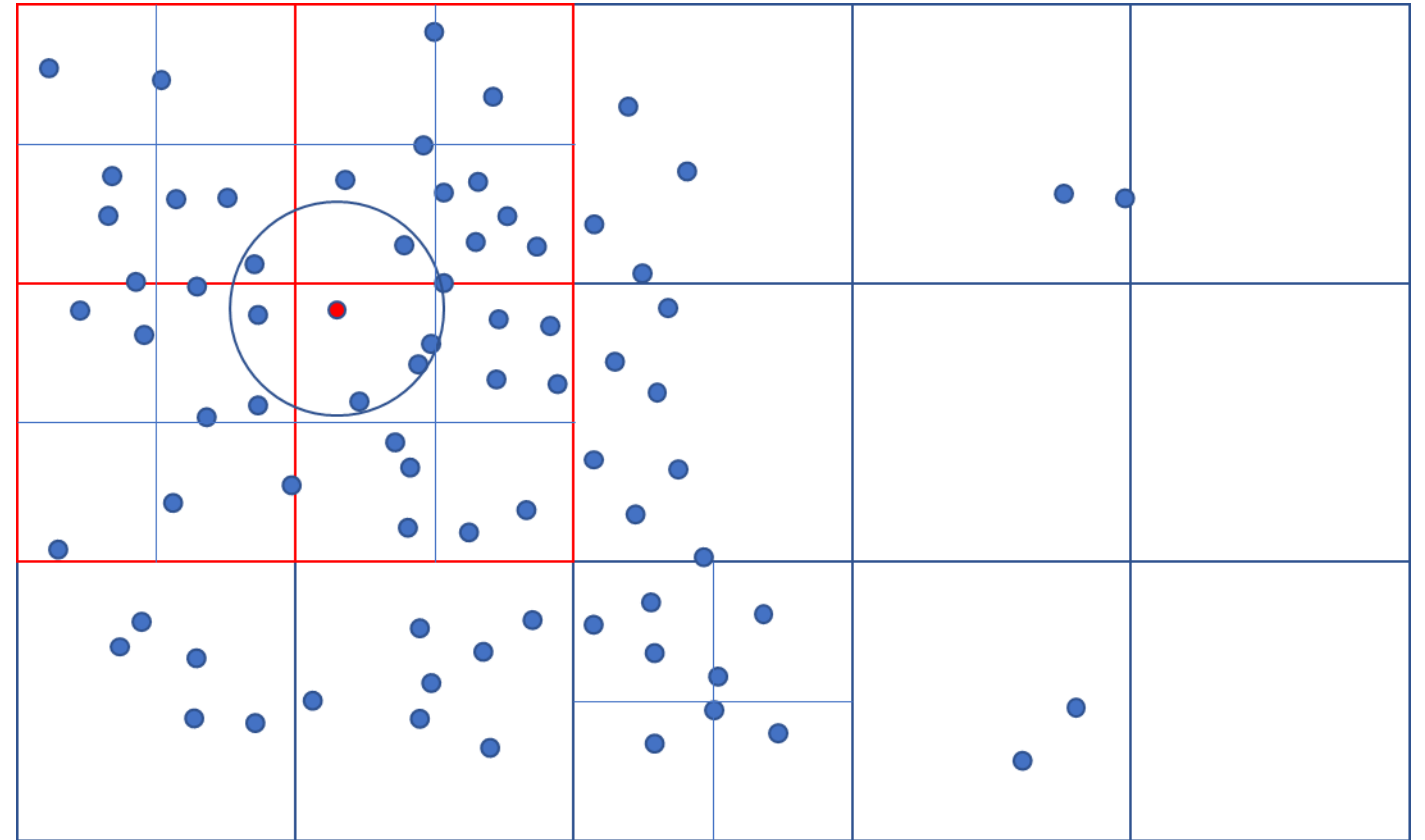
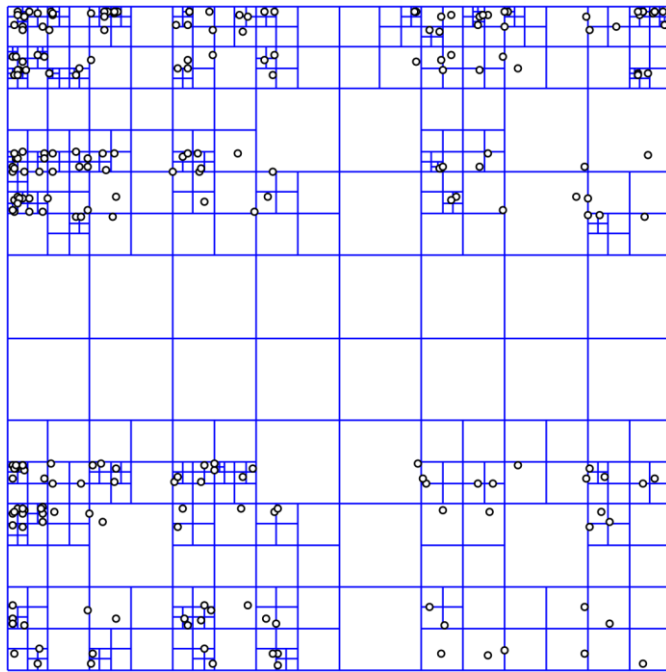
Quadtree

Quadtree

- Instead of splitting nodes into 2 regions are split into 4 regions
- Split volumes are all the same size as their siblings
- Again driven by level geometry – more populated areas are split more

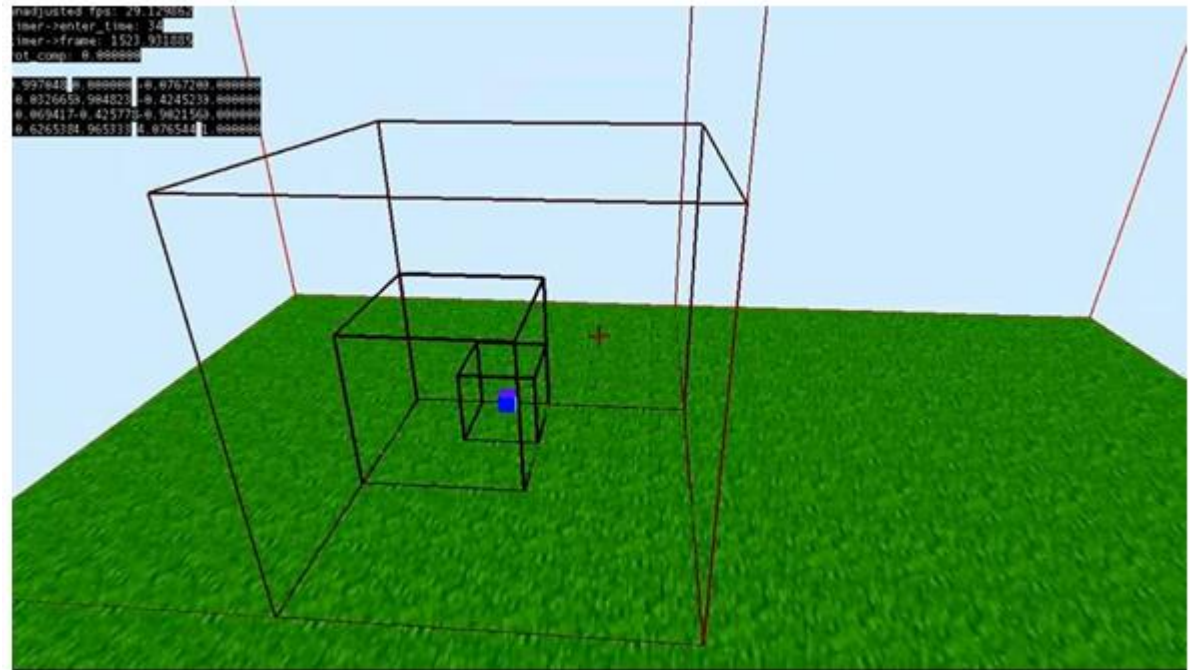


Quadtree



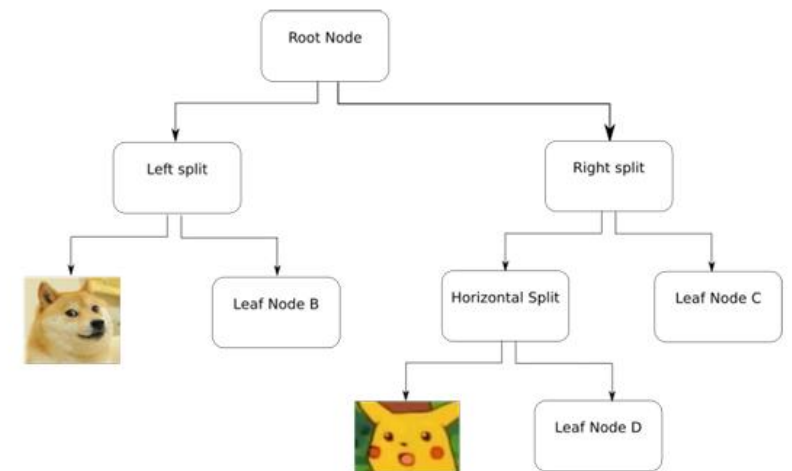
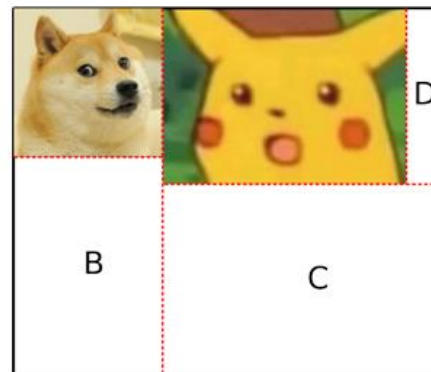
Octree – 3d version

- Divides a 3d volume into cubes
- Each 3d volume is divided into 8 regions – each axis being split in two



kDTree

- Can split the world into any number of dimensions
- Like a BSP tree each node splits into two areas
- Can change the “dimension” of the splits
 - Can split using any sort of binary check
 - AABBs are popular because they make it easier to rebuild/balance the tree



Spatial Segmentation Considerations

- Size of partitions compared to size and speed of objects
- Static or dynamic environments
 - Moving objects in space
 - Inserting new objects
 - Allowing objects to span regions
- Memory Usage